OS实验环境搭建手册

> **版本**: 1.0.0
> **建议完成时间**: 开课 / 上机前
>
> 如果你想在开课前自己准备好本地实验环境。请参考本手册。
>
> 全部准备完成后，在终端里执行 `make run` 能看到 xv6 的 shell 提示符 `-> / $` 。
>
> 恭喜你！准备已成功！
>
> 配套文档：
>
> - `lab0/实验指导.md` —— 实验零完整实验流程

目录

1. 三种环境搭建方案
2. 准备工作清单（先看这里）
3. 硬件最低配置
4. 软件清单总览
5. 本地环境搭建
6. 云端方案（希冀平台）
7. 常见问题
8. 与 lab0 的关系

一、三种环境搭建方案

根据自身条件任选其一即可：

方案	具体做法	适合谁
方案一云实验	登录希冀平台（course.educg.net），使用云端桌面 / 终端环境，所有软件与实验仓库都在里面，本地无需安装任何东西。	不方便搭本地环境；macOS（Apple 芯片）、内存8 GB 以下。
方案二导入镜像	安装 VMware Workstation 后，直接导入我们提供的 Ubuntu 24.04 预装虚拟机（稍后提供，6 GB 以上），工具链、编辑器、Claude Code、QEMU 均已装好。	想跳过安装、直接开始实验；Windows / Linux， 内存 16 GB 及以上（8GB可以，稍卡）。
方案三自己搭	自己安装 VMware Workstation、创建 Linux 虚拟机、下载 Ubuntu 24.04 官方镜像安装，再装齐交叉编译工具链、编辑器、Claude Code、QEMU 等软件，并获取实验代码仓库。	想完整体验搭建过程；Windows / Linux， 内存 16 GB 及以上（8GB可以装，稍卡）。

三种方案最终达到的效果一致：拿到一个能编译、能运行、开发 xv6-k210 OS的 Ubuntu 环境。

- 方案一走**云端方案**（见第六章），无需本地安装任何软件；
- 方案二、方案三都走**本地环境搭建**（见第五章），区别在于方案二导入预装镜像后可跳过工具链安装步骤。

二、准备工作清单

开课前请按顺序完成以下事项，每完成一项打个勾：

- ☐ 电脑满足硬件最低配置（见第三章）
- ☐ 已安装 VMware Workstation Pro 17.5.1+，且 BIOS 已开启虚拟化（此项适合本机为Windows用户使用）
- ☐ 已导入课程 Ubuntu 虚拟机镜像并能正常登录（如果我们提供的镜像，密码为123456）
- ☐ 虚拟机内已安装编辑器（VSCode 或 Cursor 二选一）
- ☐ 已安装 Claude Code 并能成功调用 Claude
- ☐ 已安装 RISC-V 交叉编译工具链（`riscv64-unknown-elf-gcc`）
- ☐ 已安装 QEMU（`qemu-system-riscv64` ≥ 7.0）
- ☐ 在 `lab0/` 下执行 `make build`、`make fs`、`make run` 全部成功，看到 `-> / $`
- ☐ 在 xv6 shell 里执行过 `ls`、`cat README`

如果你不想在自己电脑上配置环境，可以选择**希冀平台云端环境**（见第六章），跳过本地安装步骤。

三、硬件最低配置

以下配置针对**你本地电脑（宿主机）**，因为 VMware 虚拟机 + 虚拟化 + 交叉编译同时运行会占用较多资源。

项目	最低要求	建议配置	说明
CPU	x86_64，4 核，支持硬件虚拟化	8 核及以上	必须支持虚拟化：Intel VT-x 或 AMD-V
内存	8 GB	16 GB 及以上	虚拟机分配 4 GB 后，宿主机还要留出系统余量
磁盘	60 GB 可用空间	120 GB SSD	镜像 + 虚拟机磁盘 + 工具链约需 40~60 GB
操作系统	Windows 10/11 64 位，或 Linux	Windows 11 / Ubuntu 24.04	macOS：Intel 芯片可用 VMware Fusion；M 系列芯片不支持，请用云端方案或 UTM
网络	能正常访问外网	宽带	下载镜像、安装工具、调用 Claude 都需要
虚拟化开关	BIOS/UEFI 中开启 VT-x/SVM	—	不开启则 VMware 无法创建虚拟机

怎么确认电脑支持虚拟化

Windows：任务管理器 → 性能 → CPU → 查看"虚拟化"一栏是否为"已启用"。

Linux：

```
grep -E 'vmx|svm' /proc/cpuinfo | head -1
```

有输出即支持（`vmx` 是 Intel，`svm` 是 AMD）。若显示未启用，重启进 BIOS/UEFI，在 CPU 设置里打开 Intel Virtualization Technology 或 SVM Mode。

> 常见误判：内存只有 4 GB 的电脑装 VMware 后"虚拟机开不动"或"宿主机卡死"。这不是 VMware 的问题，是硬件不达标。请按上表升级，或改用云端方案。

四、软件清单总览

软件	用途	版本要求	安装位置
VMware Workstation Pro	运行 Ubuntu 虚拟机	17.5.1 及以上	宿主机（Windows/Linux）
Ubuntu	实验操作系统	24.04 LTS	虚拟机内
VSCode 或 Cursor	代码编辑器	最新稳定版	虚拟机内（推荐）或宿主机
Claude Code	AI 教学辅助	最新版	虚拟机内
RISC-V 工具链	交叉编译内核	`riscv64-unknown-elf-gcc`	虚拟机内
QEMU	模拟 RISC-V 机器	≥ 7.0	虚拟机内

> 本课程代码全部在**虚拟机内的 Ubuntu** 中编译和运行。编辑器装在哪里都可以，但推荐装进虚拟机，方便 Claude Code 直接读写课程代码。

五、本地环境搭建

> 本地环境搭建对应第一章的**方案二**和**方案三**。两种方案最终都得到一个装好全部工具的 Ubuntu 虚拟机；方案二导入预装镜像后，可跳过 5.5、5.6 的工具链与 QEMU 安装（镜像已预装）。

5.1 第 1 步：安装 VMware Workstation Pro

版本说明

课程要求 **VMware Workstation Pro 17.5.1 及以上**。该版本对个人使用免费（需要免费注册一个 Broadcom 账号后从官方渠道下载），安装后无需付费密钥。

如已安装其他版本：

- 17.5.1 及以上：可直接使用

- 17.5.0 及以下：建议升级到 26H1u1

安装步骤

1. 获取 VMware Workstation Pro 26H1u1 的安装包（`VMware-Workstation-Full-26H1u1-25688693.exe`，约 300 MB），二选一：
   - **课程网盘（推荐）**：https://pan.baidu.com/s/1Q1BzmTO4-4iQ7Iw3yok9lQ?pwd=1234 （提取码：`1234`）
   - **VMware 官方渠道**：登录VMware官网，注册/登录博通账号，并选择VMware workstation产品（Mac选择VMwarefusion）（个人使用免费，无需密钥）
2. 双击安装包，按默认选项一路下一步。
3. 安装过程中可勾选"增强型虚拟键盘驱动程序"（默认即可）。
4. 安装完成后重启电脑。
5. 首次启动 VMware 时会询问是否输入许可证，选择个人使用即可（26H1u1 个人版免费，无需密钥）。

安装前必须确认

- ☐ 第三章硬件表中"虚拟化"已开启；
- ☐ Windows 的"Windows 虚拟机监控程序平台"和"虚拟机平台"功能未冲突（若开启过 Hyper-V/WSL2，遇到 VMware 无法启动时按第七章常见问题处理）。

5.2 第 2 步：准备 Ubuntu 虚拟机

（1）两种方式（任选其一）

方式	说明	适用场景
导入实验虚拟机镜像	上课前发布课程网盘，提供装好的镜像，VMware 直接导入，环境已配好	推荐新手
全新安装 Ubuntu 及实验环境	自己新建虚拟机并安装 Ubuntu 24.04，再按本手册 5.3~5.6 手动装环境	想完整体验环境搭建过程

（2）导入课程镜像（推荐）

1. 上课前发布课程网盘，下载镜像文件压缩包到磁盘充足的分区。
2. 打开 VMware 菜单 `文件 → 打开` 选择该文件。
3. 命名虚拟机（如 `xv6-lab` ）。
4. 虚拟机硬件配置：

项目	建议值
处理器	2 核及以上
内存	4 GB 及以上
网络适配器	NAT（默认）

5. 启动虚拟机，账号密码123456。

> 若导入后虚拟机内无法联网（网络图标问号），处理：`编辑 → 虚拟网络编辑器 → 更改设置 → 重置网络` 。

（3）全新安装 Ubuntu 24.04（可选）

若选择自己装，使用国内快速下载的仓库：

（1）清华tuna镜像仓库地址

https://mirrors.tuna.tsinghua.edu.cn/ubuntu-releases/24.04.5.1/

（2）清华tuna镜像Ubuntu 24.04.5.1下载直达链接

https://mirrors.tuna.tsinghua.edu.cn/ubuntu-releases/24.04.5.1/ubuntu-24.04.5.1-desktop-amd64.iso

以上链接直接复制到浏览器或是下载器就可以直接进行下载：

1. VMware 菜单 `文件 → 新建虚拟机` → 典型。
2. 选择**「安装程序光盘映像文件(iso)」**，浏览并选中刚下载的 Ubuntu 24.04 ISO 文件。
3. 按向导填写用户名/密码、虚拟机名称和存储位置，分配 2 核 / 4 GB 内存 / 40 GB 磁盘。

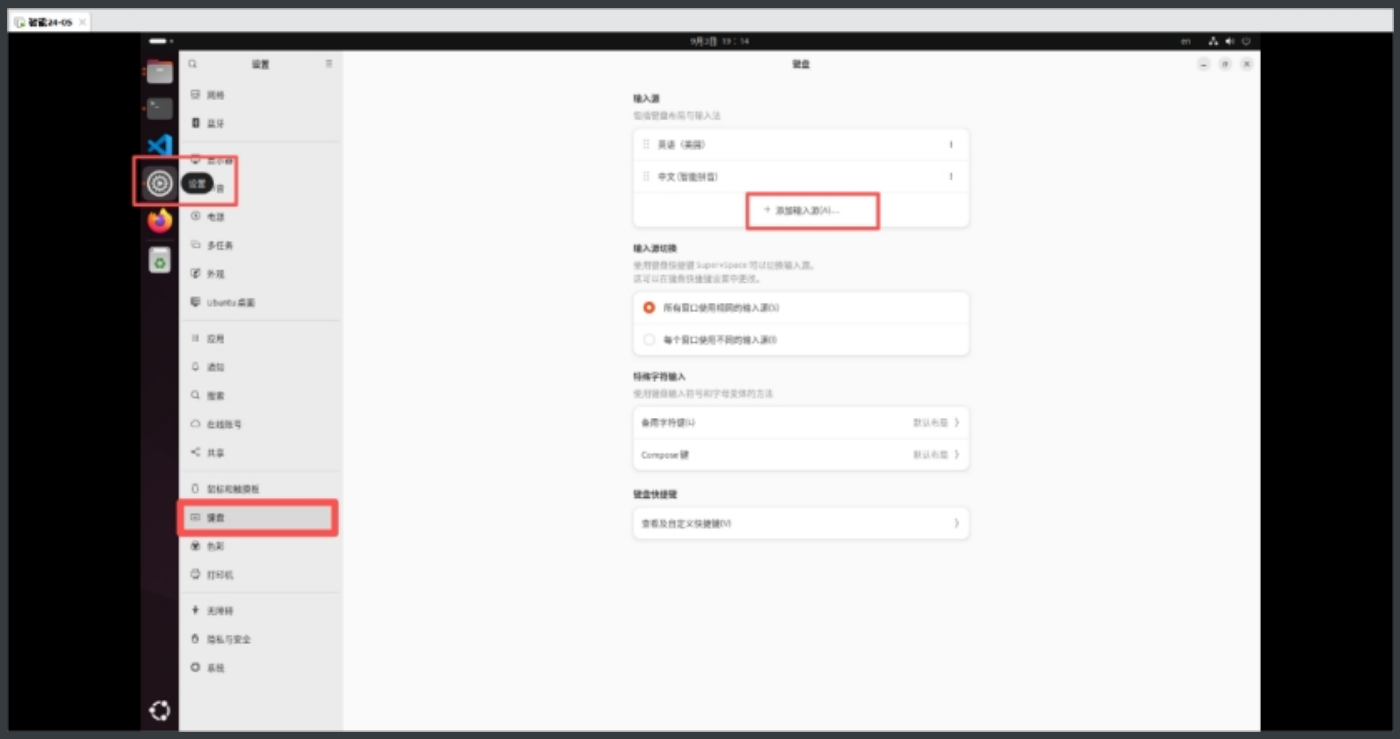
4. 点击完成后 VMware 会从 ISO 启动，进入 Ubuntu 安装向导，按提示完成安装。
5. 安装完成后打开终端（`Ctrl+Alt+T`），按 5.3~5.6 手动安装工具。

（4）登录Ubuntu后先做的事

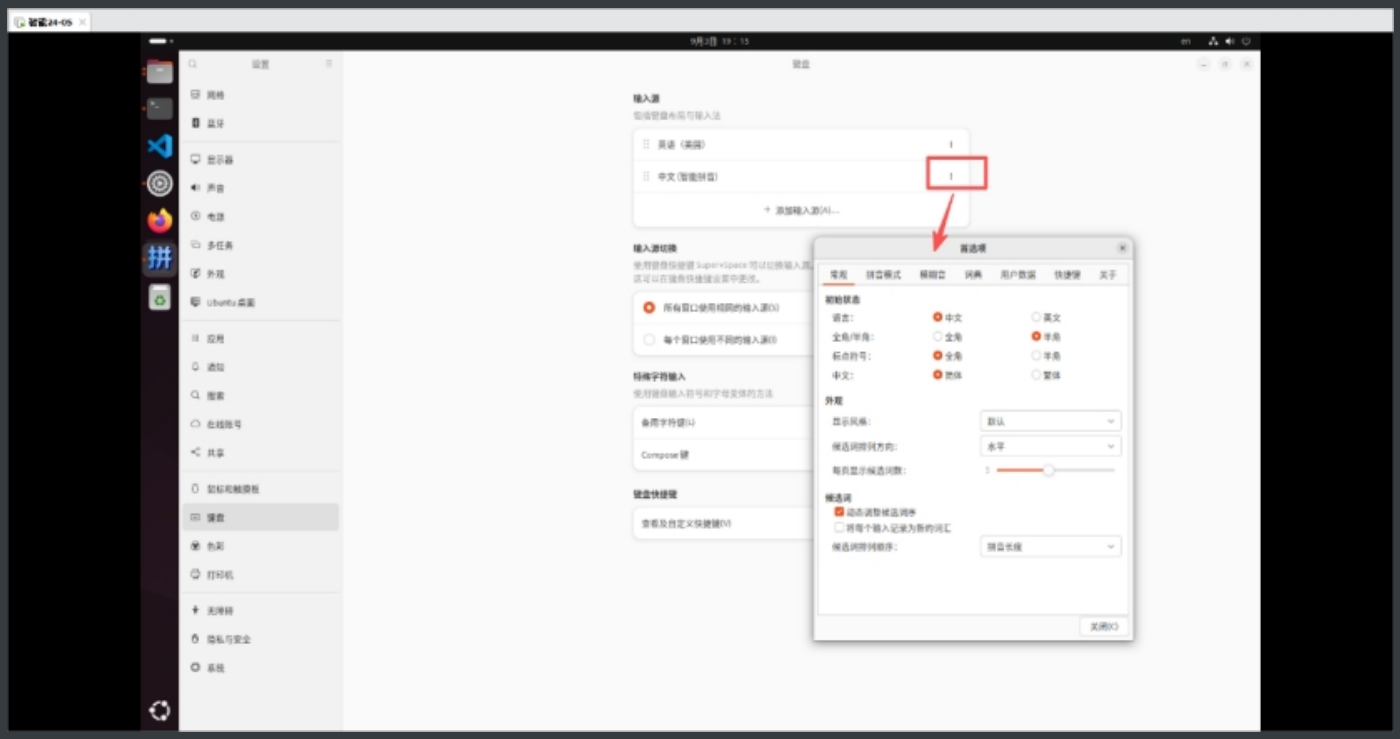
```
# 更新系统软件包（刚装好的系统建议先做）
sudo apt update
sudo apt upgrade
```

（5）配置中文输入法

点击屏幕最右上角展开面板，点击设置，切换到「键盘」选项，添加「输入源」，添加「中文（智能拼音）」



添加完成后还可以点击右侧三个小圆点对输入法进行个性化设置



（6） 安装open-vm-tools

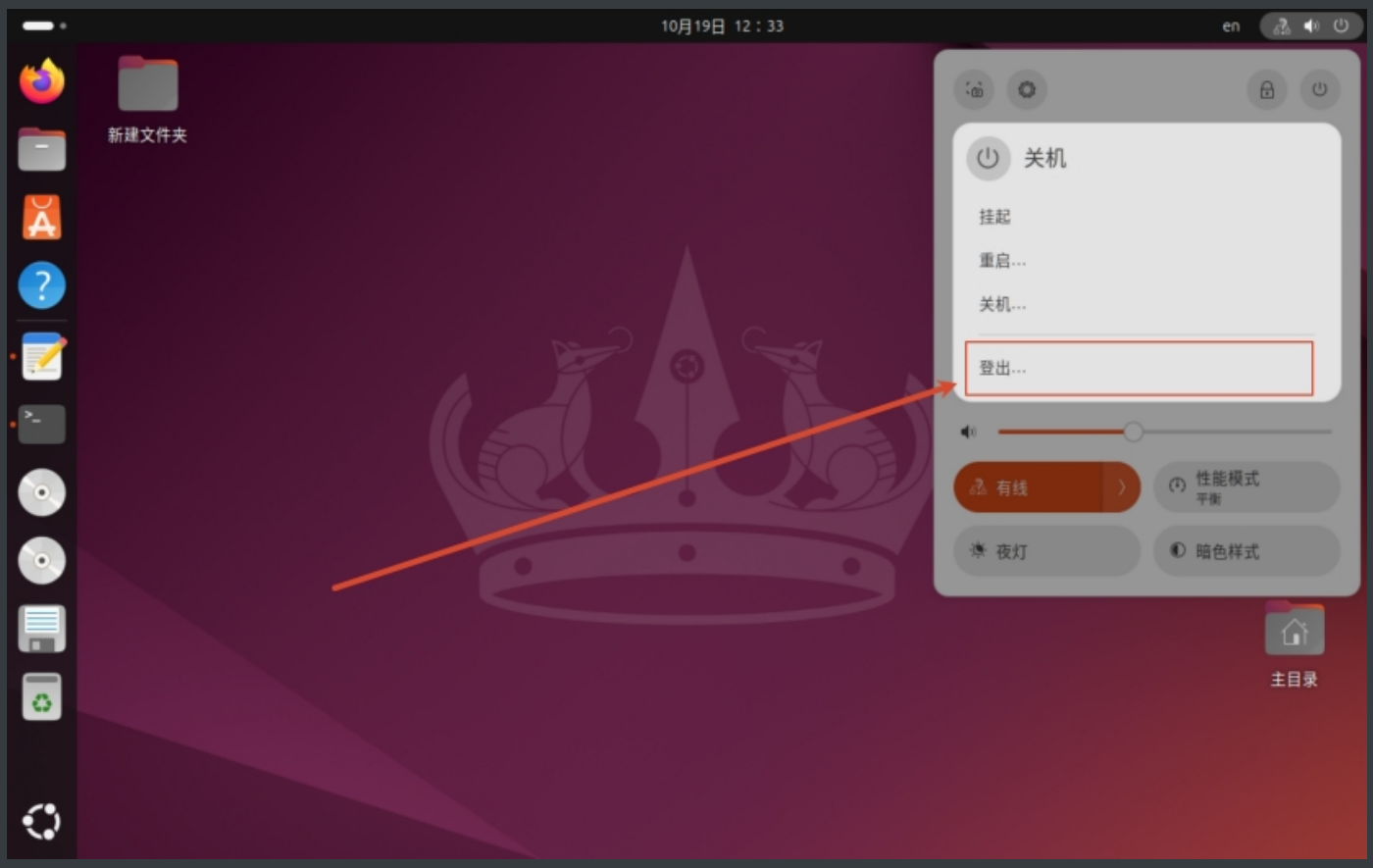
vmware虚拟机提供了vmware-tools，可以在windows主机和虚拟机之间同步剪切板，方便内容交换。

ctrl+alt+t打开终端，输入:

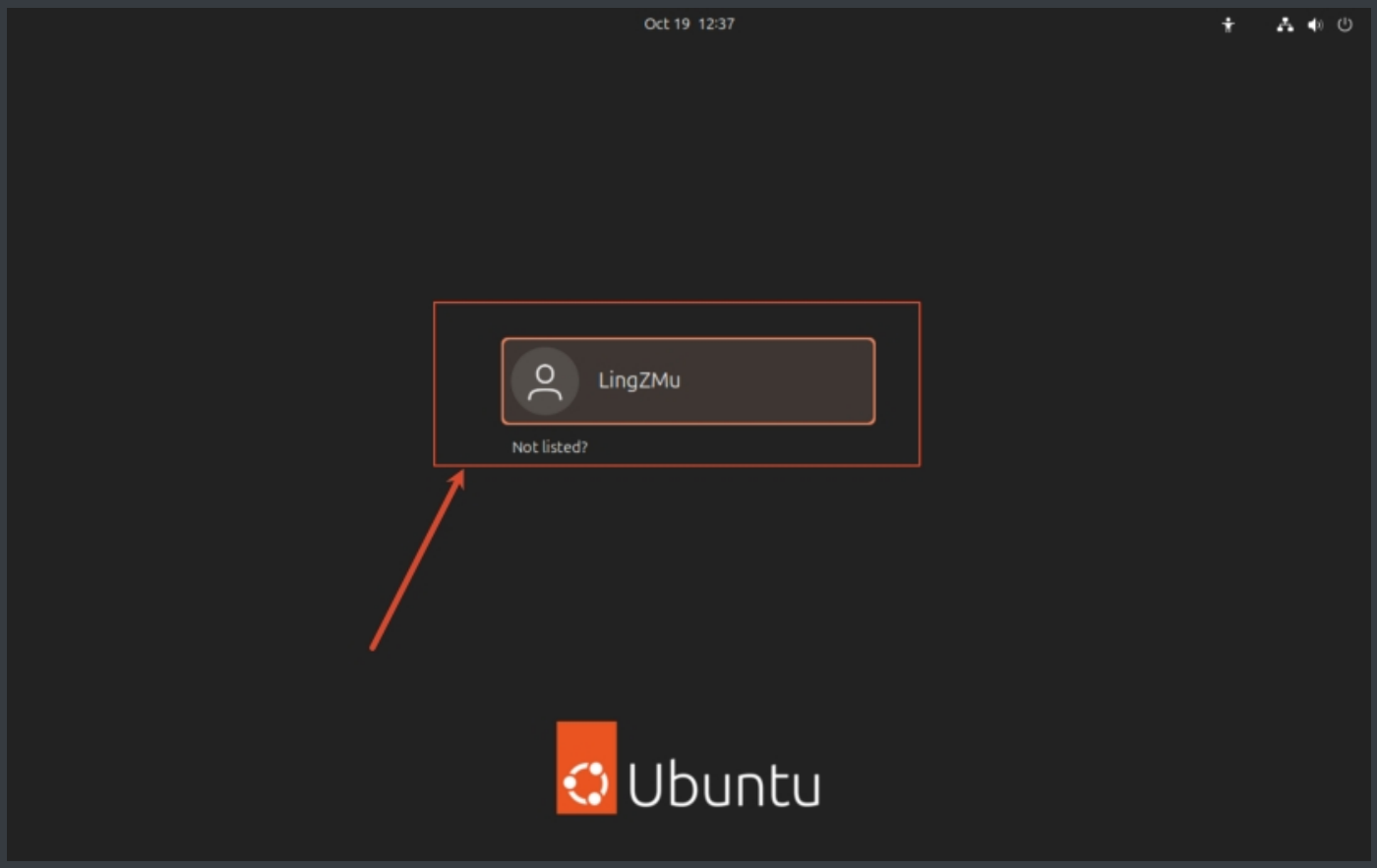
```
1   sudo apt install open-vm-tools`
```

由于vmware-tools在wayland协议下无法生效，因此需要切换到兼容x11的模式，因此仅安装完成是不行的（虽然vmware-tool在安装完以后默认立即启动），我们还需要切换系统到xrog模式

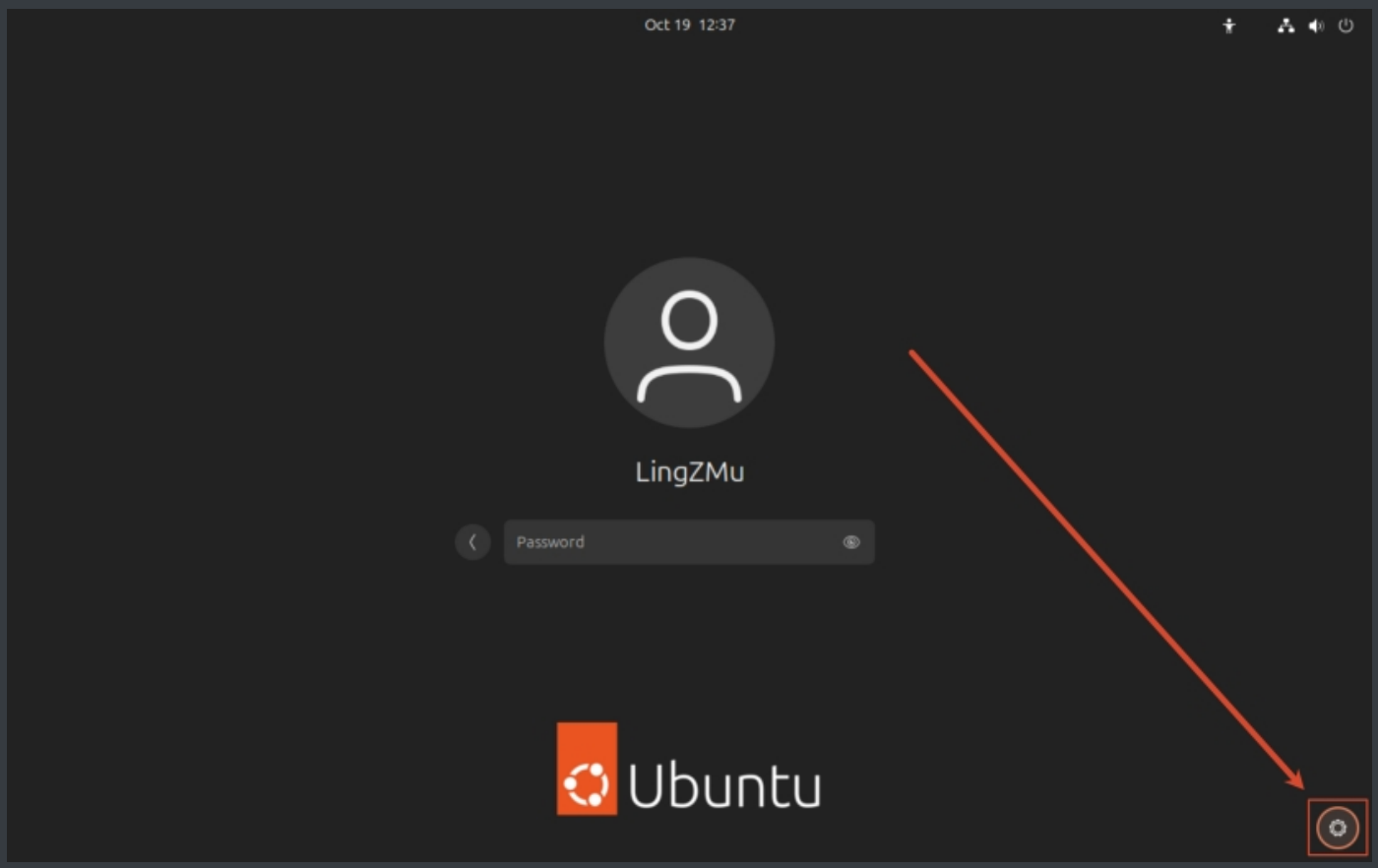
注销账户进行不那么正式的“关机”，点击右上角电源按钮，选择登出



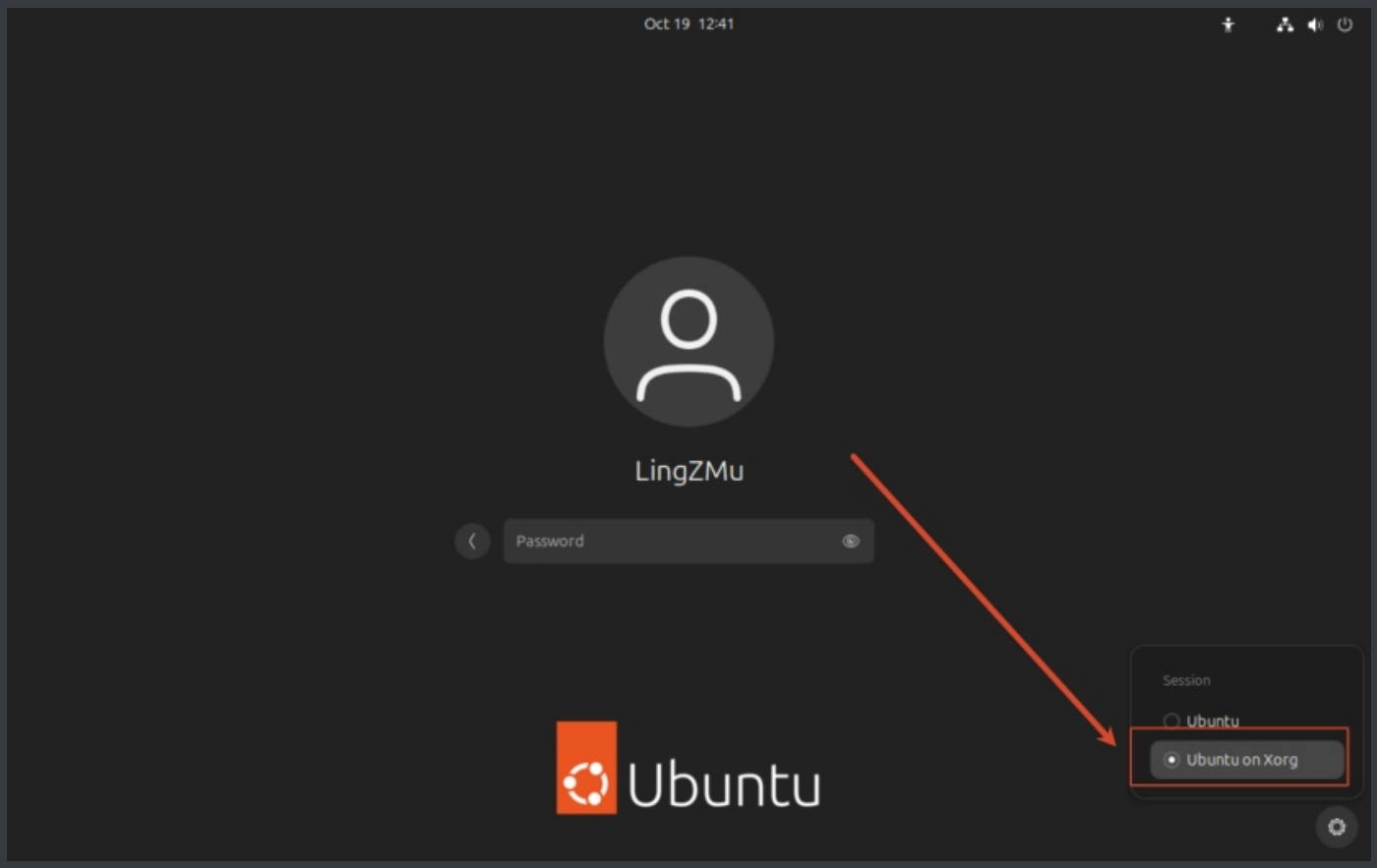
然后点击你的用户进行登入



在输密码页面注意到右下角有个齿轮按钮，点击



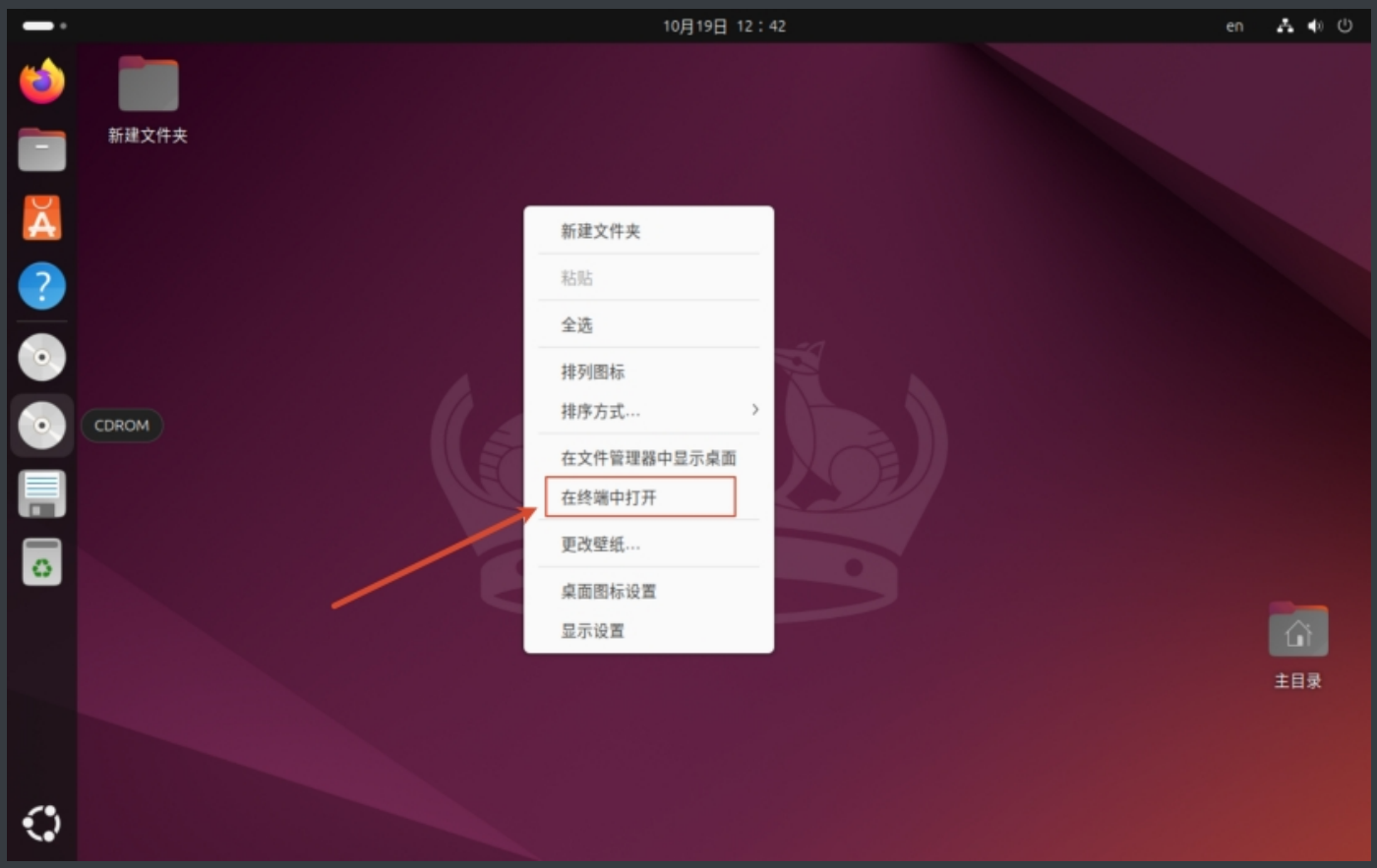
切换到ubuntu on xrog



随后输入密码正常登录系统

进入系统后测试剪切板同步是否已经生效

右键桌面-在终端中打开



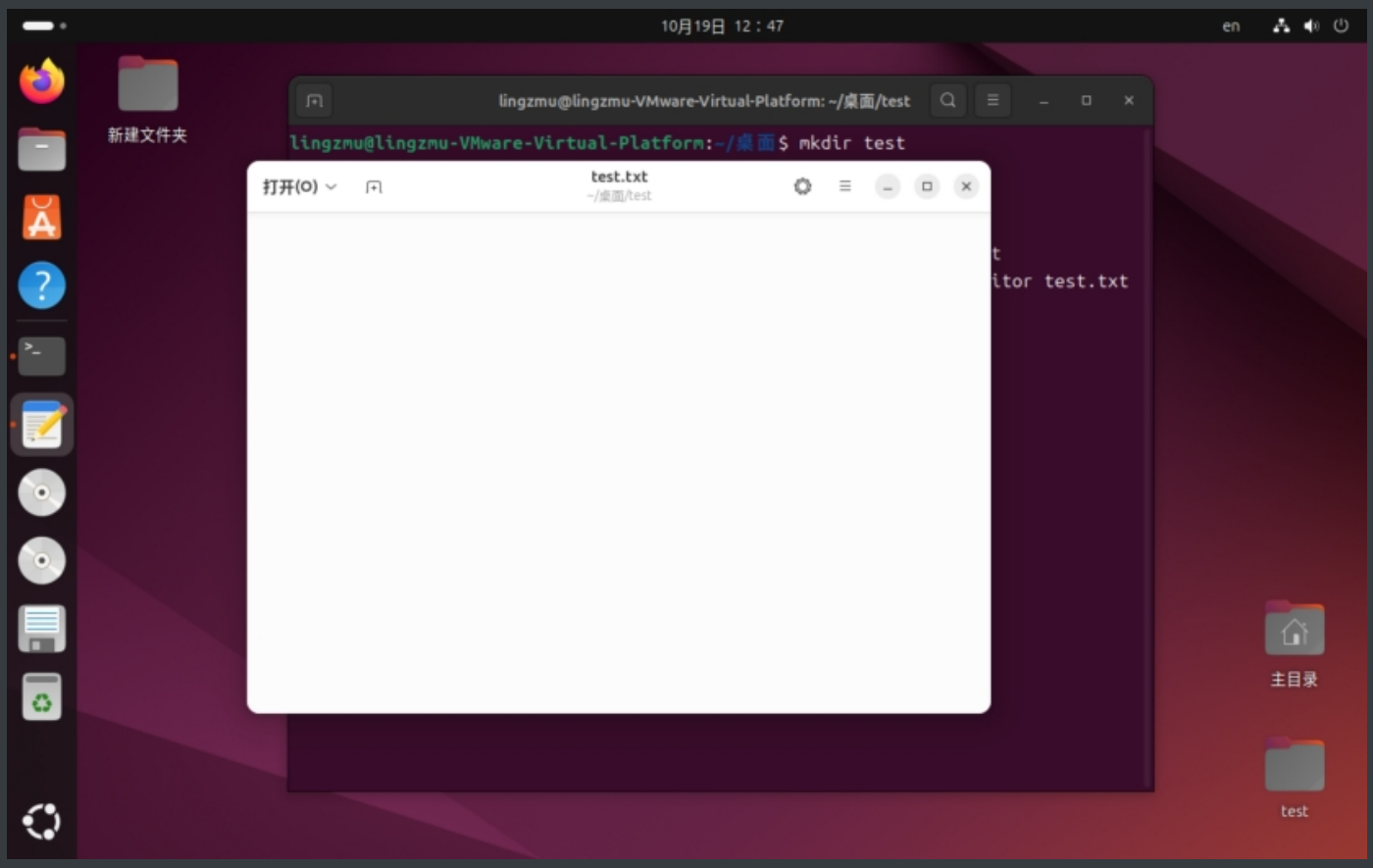
在终端中输入：

mkdir test #在桌面创建test目录

cd test #切换目录，进入/test/

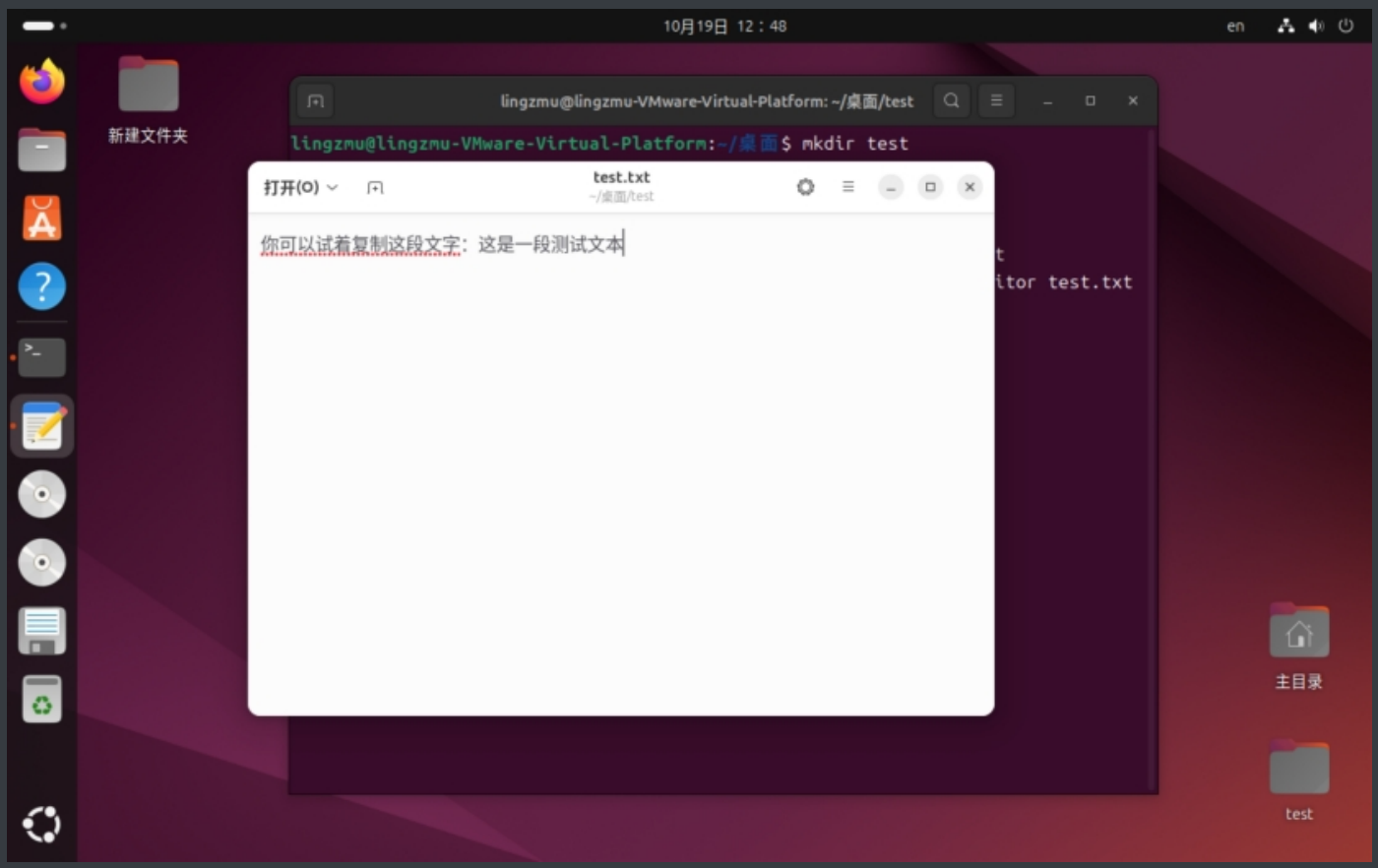
touch test.txt #新建text.txt文档

```
gnome-text-editor text.txt  #使用ubuntu自带的gnome桌面文本编辑器打开text.txt
```



然后应该会打开记事本，这个时候在windows中使用ctrl+c复制一段文字，然后进入vmware虚拟机，在记事本中使用ctrl+v粘贴，如果粘贴成功了，那么open-vm-tools就算生效了

你可以试着复制这段文字：这是一段测试文本



5.3 第 3 步：安装编辑器（VSCode / Cursor）

课程需要打开和编辑 C 源码。两者选一个即可，功能上等价，Cursor 内置了 AI 问答面板。

安装 VSCode

```
# Ubuntu 下用官方 deb 包安装
cd ~/Downloads
wget -O code.deb "https://code.visualstudio.com/sha/download?build=stable&os=linux-deb-x64"
sudo apt install ./code.deb

# 启动
code
```

也可从官网 https://code.visualstudio.com 下载安装包，或在应用商店搜索安装。

安装 Cursor

从官网 https://cursor.com 下载 Linux 版本（`.deb` 或 `.AppImage`）：

```
# .deb 方式
cd ~/Downloads
sudo apt install ./cursor-*.deb

# 或 .AppImage 方式
chmod +x Cursor-*.AppImage
./Cursor-*.AppImage
```

> Windows 宿主机上同样可以从官网下载安装包，双击安装。推荐装进虚拟机内，便于和 Claude Code 协作。

建议安装的 VSCode 扩展

- **C/C++**（Microsoft）：C 代码高亮、跳转、调试
- **Chinese Language Pack**（可选）：中文界面

5.4 第 4 步：安装 Claude Code

Claude Code 是本课程 AI 辅助学习模式的核心工具。在 `labN/` 目录下启动后，它会自动按课程约束引导你完成实验。

> **推荐路径**：直接运行课程一键脚本（见下文"配置 API 与中转地址"方式一），它会自动完成 Node.js 安装、Claude Code 安装和密钥配置。下面是手动拆解步骤，供想自己控制每一步的读者参考。

安装 Node.js（≥ 18，推荐 22 LTS）

```
node -v   # 检查是否已有 Node；若 ≥ v18 可跳过本步
```

如果没有，用 **nvm** 安装 Node 22 LTS（比 apt 仓库自带的版本新，且按用户安装、不污染系统、可随时切换）：

```
curl -fsSL https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.3/install.sh | bash
export NVM_DIR="$HOME/.nvm"; [ -s "$NVM_DIR/nvm.sh" ] && . "$NVM_DIR/nvm.sh"
nvm install 22 && nvm use 22
node -v   # 应输出 v22.x
```

> 为什么不用 `apt` 装 Node：Ubuntu 24.04 仓库里的 Node 版本偏旧，且升级麻烦；Claude Code 要求 Node ≥ 18，推荐 22 LTS。

安装 Claude Code

```
npm install -g @anthropic-ai/claude-code

# 验证
claude --version
```

配置 API 与中转地址（两种方式任选其一）

方式一：课程一键脚本（推荐）

```
curl -O "https://kuafuai.art/claude_code_env.sh" && bash ./claude_code_env.sh
```

脚本自动完成 6 步：

1. 检查/安装 Node.js（nvm 装 Node 22 LTS，要求 ≥ 18）；
2. 安装 Claude Code（`npm install -g @anthropic-ai/claude-code`）；
3. 初始化 Claude Code 状态，跳过首次启动的交互向导；
4. 引导你输入**中转地址**与**API Key**（输入时不回显，不落 shell 历史）；

5. 从中转站的 `/v1/models` 拉取可用模型列表，用 ↑/↓ 选择要用的模型；
6. 选择是否使用 **1m 长上下文版本**，然后把配置写入 `~/.claude/settings.json`。

配置完成后直接运行 `claude` 即可，**无需每次 export**。

方式二：手动配置（不想跑脚本时）

安装完成后，编辑 `~/.claude/settings.json`（不存在就新建），写入：

```
{
  "env": {
    "ANTHROPIC_AUTH_TOKEN": "sk-...",
    "ANTHROPIC_BASE_URL": "https://<模型url>",
    "ANTHROPIC_MODEL": "<模型名>",
    "ANTHROPIC_SMALL_FAST_MODEL": "<模型名>",
    "API_TIMEOUT_MS": "3000000",
    "CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC": "1"
  }
}
```

字段说明：

- `ANTHROPIC_AUTH_TOKEN`：API Key 用 **token 方式**传入，Claude Code 会自动读取，**不要用 `export ANTHROPIC_API_KEY` 写进环境变量**——既不持久（新终端失效）、还会留在 shell 历史里；
- `ANTHROPIC_BASE_URL`：中转 API 地址，由课程教师统一提供（或使用大模型厂商密钥）；
- `ANTHROPIC_MODEL` / `ANTHROPIC_SMALL_FAST_MODEL`：主模型与轻量模型，一般填同一个模型名（脚本第 5 步会从 `/v1/models` 自动列出可用模型，选 1m 版本时模型名后带 `[1m]` 后缀）；
- `API_TIMEOUT_MS`：请求超时（毫秒）。xv6 编译/跑测试耗时长，默认超时容易中断，这里设为 3000000（50 分钟）；
- `CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC`：关闭非必要网络流量。

网络注意事项（重要）

Claude 官方 API 在国内访问可能不稳定。如果遇到连接失败或超时，确认：

1. 中转地址是否正确写入 `~/.claude/settings.json`（不是 shell 里 export 的临时变量）；
2. 中转地址和模型名是否与教师提供的一致（脚本方式会自动拉取模型列表，不易填错）；
3. 仍超时的话，检查 `API_TIMEOUT_MS` 是否被覆盖为小值。

验证 Claude Code 可用

```
cd lab0
claude
```

进入后输入一句"你好"，能正常回复即表示配置成功。输入 `/exit` 退出。

5.5 第 5 步：安装本地gcc编译调试工具和 RISC-V 交叉编译工具链

（1）安装intel本地gcc编译调试工具

安装C语言基础编译工具

sudo apt install build-essential

安装GDB调试工具

sudo apt install gdb

（2）安装RISC-V 交叉编译工具链

xv6 内核运行在 RISC-V 架构上，但我们是在 x86_64 的电脑上编译它，所以需要交叉编译器。

安装

```
# 检查是否已装
riscv64-unknown-elf-gcc --version

# 未安装则用 apt 安装
sudo apt install gcc-riscv64-unknown-elf
```

确认 TOOLPREFIX 与工具链匹配

打开实验仓库 `lab0/Makefile`，找到：

```
TOOLPREFIX  := riscv64-unknown-elf-
# TOOLPREFIX  := riscv64-linux-gnu-
```

- 装的 `riscv64-unknown-elf-gcc` → 保持默认即可；
- 装的 `riscv64-linux-gnu-gcc`（`sudo apt install gcc-riscv64-linux-gnu`）→ 把前缀改为 `riscv64-linux-gnu-` 并注释掉原行。

`TOOLPREFIX` 与工具链不匹配会直接编译失败。

验证

```
riscv64-unknown-elf-gcc --version
riscv64-unknown-elf-ld --version
```

两条命令都能输出版本号即可。

> GCC 13 兼容说明：Ubuntu 24.04 的 `riscv64-unknown-elf-gcc` 通常是 GCC 13，对 `sh.c:runcmd()` 会报 `infinite recursion detected` 警告。本课程仓库已在 Makefile 的 `CFLAGS` 中加入 `-Wno-error=infinite-recursion`，无需处理。若看到 `LOAD segment with RWX permissions` 的链接警告，那是警告不是错误，只要最终生成 `target/kernel` 即可继续。

（3）安装交叉调试工具 gdb-multiarch

在 ubuntu 上用 gdb-multiarch配合qemu 调试 xv6-k210。
安装命令：

```
sudo apt install gdb-multiarch
```

验证：

```
gdb-multiarch --version
```

5.6 第 6 步：安装 QEMU 系统模拟器

安装

```
# 检查是否已装，期望版本 ≥ 7.0
qemu-system-riscv64 --version

# 未安装或版本过低则安装
sudo apt install qemu-system-misc
```

验证

```
qemu-system-riscv64 --version
```

期望输出形如：

```
QEMU emulator version 8.2.2
Copyright (c) 2003-2023 Fabrice Bellard and the QEMU Project
developers
```

同时安装文件系统工具

```
sudo apt install dosfstools mtools
```

`make fs` 生成 FAT32 镜像 `fs.img` 时需要 `mkfs.vfat`（dosfstools 提供）。

> 若发行版自带 QEMU 版本 < 7.0（一般不会），才需要从源码编译，步骤见 `lab0/实验指导.md` 任务 3 步骤 3。

5.7 第 7 步：获取实验仓库

每次实验的代码压缩包，会发布在希冀平台课程的云实验里。用自己账号登录希冀平台后，下载到本地，就可以在本地虚拟机里做实验了。

进入实验目录：

```
cd lab0
```

实验目录下有 2个版本实验指导书，一个详细，一个简洁，根据自己思维喜好选择。

5.8 第 8 步：运行验证清单

环境全部装好后，按 lab0 流程完整跑一遍：

```
cd lab0

# 1. 构建内核（交叉编译 + 链接）
make build

# 2. 生成文件系统镜像（首次需要 sudo 密码）
make fs

# 3. 启动 xv6-k210
make run
```

成功标志：终端依次出现 OpenSBI banner、内核输出、`hart 0 init done`、`init: starting sh`，最终出现 shell 提示符：

```
1  -> / $
```

在 shell 里试几个命令：

```
1  -> / $ ls
2  -> / $ ls /bin
```

退出 QEMU：按 `Ctrl+A`，松开后按 `X`。

验证自检表

- ☐ `riscv64-unknown-elf-gcc --version` 有输出
- ☐ `qemu-system-riscv64 --version` ≥ 7.0
- ☐ `make build` 成功，生成 `target/kernel`
- ☐ `make fs` 成功，生成 `fs.img`（约 256 MB）
- ☐ `make run` 进入 `-> / $`
- ☐ xv6 shell 中 `ls`、`cat README` 正常
- ☐ `claude` 在 `lab0/` 目录下能正常对话

全部打勾 = 环境准备完成，可以开始 lab0 的学习。

六、云端方案（希冀平台）

> 本节对应第一章**方案一**。不方便搭建本地环境、或进行远程教学时使用。

登录希冀平台，选择本课程云实验，即可获得已配好全部工具的实验环境。

- **终端界面模式**：通过平台提供的终端直接操作，体验与本地终端一致。
- 具体登录地址、镜像选择和操作步骤参考《操作系统基础训练营希冀平台指南.pdf》。

七、常见问题

现象	原因 / 解决
VMware 无法启动虚拟机	BIOS 未开启虚拟化，或与 Hyper-V/WSL2 冲突。开启 VT-x/SVM 后重启；Windows 上可尝试关闭"虚拟机监控程序平台"或改用 VMware 26H1u1
虚拟机内无法联网（网络图标问号）	VMware 虚拟网络适配器映射错位。`编辑 → 虚拟网络编辑器 → 更改设置 → 重置网络`，重启虚拟机
虚拟机运行卡顿	宿主机内存不足（需 8 GB 以上）；给虚拟机降到 2 核/4 GB；关闭宿主机其他大程序
`riscv64-unknown-elf-gcc: command not found`	工具链未安装，执行 `sudo apt install gcc-riscv64-unknown-elf`
`make build` 报 `unrecognized command-line option`	工具链版本与内核期望不匹配，或 TOOLPREFIX 不对，检查 `Makefile`
`make build` 报 `infinite recursion detected`	GCC 13 兼容性诊断；确认 Makefile 含 `-Wno-error=infinite-recursion`
`qemu-system-riscv64: command not found`	QEMU 未安装，执行 `sudo apt install qemu-system-misc`
`make fs` 报 `mount: operation not permitted`	没用 sudo，或 `/mnt` 被占用。先 `sudo umount /mnt` 再重试
`make run` 后看到 `->` / `$` 但输入无反应	QEMU 8.2/OpenSBI 1.3 与旧控制台输入不兼容；当前仓库已修复，退出旧 QEMU 后重新 `make run`
`claude` 连接失败或超时	网络问题。配置代理或按 5.4 节使用教师提供的中转地址
`npm` 命令找不到	Node.js 未安装或 PATH 未生效，重开终端或重新安装 Node 20

八、与 lab0 的关系

本手册是"开课前准备"，lab0 实验指导是"上课内容"：

事项	手册	说明
硬件与软件准备	本手册（准备）	开课前完成
三种方案选择	本手册第一章	本地（方案二/三）或云端（方案一）
安装工具链 / QEMU	本手册 5.5、5.6 + `实验指导.md` 任务 2、3	命令一致，实验指导讲原理
构建 / 运行 / 验收	`实验指导.md` 任务 4~7	上课完成

> 如果你使用课程**实验镜像**（已预装工具链和 QEMU），本手册 5.5、5.6 可跳过，直接进入 5.7 获取仓库并运行验证。